

Оптимизация LLM-инференса на Эльбрус 8С2

Платформа:

Конфигурация:

Память:

Компилятор:

Модель:

Дата:

Репозиторий:

HEAD:

Содержание

1. Резюме результатов

Конфигурация	Lossless		
	✓	3.3	1.0×
	✓	5.2	1.6×
PT_Q8_SOA=1	✓	9.4	2.8×
	✓	10.6	3.2×
	✓		
PT_LAYER_SKIP="22,24,26,28,30,32"	✗	13.2	4.0×
PT_LAYER_SKIP="12,14,...,34"	✗	15.5	4.7×

Главный результат.	lossless потолок 11.4 tok/s
--------------------	-----------------------------

Сравнение	
	164.7
	82.6
Эльбрус 8C2 / PromeTorch TP-4 lossless (тот отчет)	
	15.5
	3.3

2. Per-section профайл (90 ms breakdown)

PT_Q8_SOA=1 PT_PROFILE_LAYER=1
" --max-tokens 100 "

Секции			Содержимое
	25.69	28.6%	
	20.39	22.7%	
	17.47	19.4%	
	16.37	18.2%	
	4.73	5.3%	
	3.64	4.0%	
	1.61	1.8%	
	0.06	0.1%	
ИТОГО			

Распределение нагрузки:

- 71%
- 19%
- 6%
- 4%

▷

3. Q8 SoA4 weight layout — корневой kernel под qpmaddubsh

3.1 Структура блока 176 байт

`__builtin_e2k_qpmaddubsh`

Per super-row block (4 N-rows × 32 K-elements = 176 байт):

bytes 0..15 : 4× fp32 d_w (per-row block scale = d × sub_scale_q4k)

bytes 16..31 : 4× fp32 dmin_m (per-row min = dmin × sub_min_q4k)

bytes 32..47 : 4× int32 sum_q (per-row sum of int8 weights в этом блоке)

bytes 48..175 : 128 байт byte-interleaved weights:
8 K-групп × 16 байт каждая, layout per K-group of 4 K-elem × 4 rows:
[r0[0..3], r1[0..3], r2[0..3], r3[0..3]]

3.2 Внутренний K-loop и арифметика fold

```
v2di acc_i32 = {0, 0};
// Inner: 8 K-групп × 4 элемента = 32 элемента всего
_Pragma("loop count(8)") _Pragma("ivdep")
for (int kg = 0; kg < 8; kg++) {
    v2di p16 = __builtin_e2k_qpmaddubsh(W_v[kg], A_v[kg]); // 8×i16 partial
    v2di p32 = __builtin_e2k_qpmaddh(p16, ONES16);          // 4×i32 reduce
    acc_i32 = __builtin_e2k_qpaddw(acc_i32, p32);
}

// Per-block FP fold (5 ops): dot_signed = acc - 128*sum_q
// fp_acc += scale_a * (d_w * dot_signed - dmin_m * sum_a_block)
v2di shift_v = __builtin_e2k_qpmullw(sum_q_v, SOA4_SHIFT128);
v2di dot_signed = __builtin_e2k_qpsubw(acc_i32, shift_v);
v2di acc_f = __builtin_e2k_qpistofs(dot_signed);
v2di term_w = __builtin_e2k_qpfmuls(scales_v, acc_f);
v2di term_d = __builtin_e2k_qpfmuls(dmins_v, sa_v);
v2di delta = __builtin_e2k_qpfmuls(scale_a_v,
    __builtin_e2k_qpfsubs(term_w, term_d));
fp_acc = __builtin_e2k_qpfadds(fp_acc, delta);
```

`qpmaddubsh`

-

4

`qpmaddh`

`q8_soa4_quant_activation`

3.3 Микробенч

`vliw_mission/e2k_vnni/q8_soa4_microbench.c`

Реализация		Комментарий
	1.21	
	1.03	


```

slot 0 (load weight)
slot 1 (compute)
slot 2 (AAU update)
slot 5 (load weight)

← slot 1 (compute)
← slot 2
← slot 4 (compute)
← slot 5

_f16s 0x1f0, %qpg28
_qpr34, %qpr32, %qpr27
_f16s 0x2b0, %qpg29
_r22, _f16s 0x1e0, %qpg28
i,4,sm %qpr33, %qpr27
sm %dr22, _f16s 0x1f0, %qpg28

```

Результат анализа

5.2 Persistent broadcast ThreadPool с futex

```
c10/util/ThreadPool.h
```

```
c10/util/Futex.h
```

-
-
-
-

Эффект: **+0.5 tok/s (9.4 → 9.9)**

5.3 8 потоков на rank, drop NUMA replicate

Эффект: **+0.7 tok/s (9.9 → 10.6)**

5.4 Fused gate+up Q8 SoA4 GEMV (dual)

```
q8_soa4_gemv_dual
```

Эффект: **+0.2 tok/s (10.6 → 10.8)**

5.5 AVX2/e2k SIMD attention math

```
#if defined(__AVX2__)
```

```
#elif defined(__e2k__)
```

```
qpfmuls + qpfadds
```

Эффект: **+0.6 tok/s (10.8 → 11.4)**

5.6 Fused SiLU + Q8 quant_activation

```
q8_soa4_silu_quant_activation_fused
```

```
std::vector<uint8_t>(K)
```

Эффект:

5.7 Triple-fused QKV GEMV

```
q8_soa4_gemv
```

```
q8_soa4_gemv_triple
```

Эффект:

5.8 RoPE cos/sin cache + scores buffer reuse

- `past_len`

```
rope_precompute
```

```
TPParallelState.rope_*_cache
```

- `std::vector<float>(total_seq)`

```
tp_.scores_buf
```


Эффект:

6. Опт-ин lossy режим PT_LAYER_SKIP

`x_buf[cur]`

`PT_LAYER_SKIP="i,j,k,..."`

`только в decode-фазе`

`tp.in_decode_phase`

Sweep alternating skip patterns (decode-only, 100 токенов)

Pattern			Quality output text
	0	11.4	
	6	13.2	
	8	13.8	
	9	14.4	
	10	14.7	
	11	14.9	
12,14,...,34		★	
	12	15.1	

Эмпирический вывод.

7. Эмпирически опровергнутые гипотезы

не дали обещанного speedup

7.1 Q4 SoA4 (4-bit packed) — провал на SSE2 emulation

`vliw_mission/e2k_vnni/q4_soa4_microbench.c`

```
// SSE2 nibble unpack pattern:
__m128i packed = _mm_load_si128(qs);
__m128i lo     = _mm_and_si128(packed, MASK_0F);    // 8 i8 low nibbles
__m128i hi     = _mm_and_si128(_mm_srli_epi16(packed, 4), MASK_0F);
__m128i exp    = _mm_unpacklo_epi8(lo, hi);         // interleave
```

Результат: **10.82 ms**

`_mm_srli_epi16`

`_mm_unpacklo_epi8`

7.2 Dual-accumulator unroll в k-loop — null effect

acc_i32

```
// Было: 1 chain
v2di acc_i32 = {0, 0};
for (kg = 0; kg < 8; kg++) acc_i32 = qpaddw(acc_i32, qpmaddh(qpmaddubsh(...), ONES16));

// Стало: 2 chains, summed at end
v2di acc_a = {0,0}, acc_b = {0,0};
for (kg = 0; kg < 8; kg += 2) {
    acc_a = qpaddw(acc_a, qpmaddh(qpmaddubsh(W[kg], A[kg]), ONES16));
    acc_b = qpaddw(acc_b, qpmaddh(qpmaddubsh(W[kg+1], A[kg+1]), ONES16));
}
v2di acc_i32 = qpaddw(acc_a, acc_b);
```

Результат:

7.3 FMA fold через qpfmmas/qpfmas — недоступны на v5

```
// Было (5 ops):
v2di term_w = qpfmuls(scales, acc_f);
v2di term_d = qpfmuls(dmins, sa);
v2di delta = qpfmuls(scale_a, qpfsbsh(term_w, term_d));
fp_acc = qpfadds(fp_acc, delta);

// Цель (3 ops через qpfmmas:= -(a*b)+c, qpfmas:= a*b+c):
v2di term_w = qpfmuls(scales, acc_f);
v2di diff = qpfmmas(dmins, sa, term_w); // = -(dmins*sa) + (scales*acc_f)
fp_acc = qpfmas(scale_a, diff, fp_acc); // = scale_a*diff + fp_acc
```

Результат:

-march=elbrus-v5

error: built-in function "__builtin_e2k_qpfmmas" is not supported for current cpu mode

/opt/mcst/lcc-home/1.29.16/e2k-8c2-linux/include/e2kintrin.h

7.4 __builtin_prefetch на weight blocks — null

```
for (int64_t b = 0; b < bpr; b++) {
    if (b + 2 < bpr) __builtin_prefetch(gp + (b + 2) * SOA4_GROUP_BYTES, 0, 1);
    ...
}
```

Результат:

7.5 Batched GEMM K=4 — РЕГРЕССИЯ 0.42×

q8_soa4_gemm4_microbench.c

Реализация		Комментарий
	0.707	
	6.732	
	1.683	0.42× РЕГРЕССИЯ

Корневая причина.

Implication для МЦСТ.
compute-bound
не дадут wins

7.6 Busy-spin ThreadPool (llama.cpp / Gemini DR pattern) — РЕГРЕССИЯ ×2

c10/util/ThreadPool.h

Результат: 11.4 → 5.5 tok/s (×2 регрессия)

Корневая причина.

gen_.load(memory_order_acquire)

Эмпирический вывод.
правильный

futex-based pool

7.7 PT_TP_GATHER (AllGather mode) — регрессия + не lossless

Результат: 9.0 tok/s регрессия + НЕ lossless

8. Заключение и рекомендации МЦСТ

8.1 Подтвержденный lossless потолок — 3 независимых угла

подтвержденным peak

Дисассемблирование q8_soa4_gemv

q8_soa4_gemm K=4 microbench

busy-spin pool probe

×3.5

8.2 Что было бы полезно от МЦСТ для следующих generation / LCC release

8.2.1 Backport qpfmas/qpfmmas на v5

e2kintrin.h

8.2.2 Native E2K instructions для 4-bit nibble unpack

_mm_srli_epi16 _mm_unpacklo_epi8

-
-
- `__builtin_e2k_qpsrlw_epi16(v, count)`
- `__builtin_e2k_qpand_8b(a, b)`
- `__builtin_e2k_qpunpcklbw_128(a, b)`

8.2.3 LCC и APB documentation

/opt/mcst/doc/lcc/

-
-
-

8.2.4 Cross-NUMA atomic latency documentation

-
-
-

8.2.5 Wider INT8 dot-product instruction (32-byte)

```
qpmaddubsh
```

8.2.6 SVE-style scalable vector / variable-length

9. Приложение А: команды воспроизведения

На Эльбрусе после reboot:

```
loginctl enable-linger $USER # обязательно – иначе systemd убивает процессы при SSH disconnect
```

Сборка:

```
cd ~/promethorch
cmake -S . -B build_elbrus \
  -DCMAKE_BUILD_TYPE=Release \
  -DPT_USE_CUDA=OFF -DPT_BUILD_TESTS=OFF
cmake --build build_elbrus --target test_gguf_inference -j 16
# LCC компилирует ~7 минут на gguf_model.h (6.7k строк header)
```

Запуск 4-NUMA TP-4 lossless 11.4 tok/s:

```
PT_Q8_SOA=1 bash scripts/run_tp_elbrus.sh --greedy \
  "Write a short haiku about artificial intelligence"
# Expected: 100 tokens in ~8.7s (11.4 tok/s)
```

Запуск с lossy LayerSkip 15.5 tok/s:

```
PT_Q8_SOA=1 PT_LAYER_SKIP="12,14,16,18,20,22,24,26,28,30,32,34" \
  bash scripts/run_tp_elbrus.sh --greedy "..."
# Expected: ~15.5 tok/s, output text degraded но координированный
```

Per-section профайл:

```
PT_Q8_SOA=1 PT_PROFILE_LAYER=1 bash scripts/run_tp_elbrus.sh --greedy "..."
# Печатает [prof-TP rank0] avg ms/token по 8 секциям
```

Дисассемблирование q8_soa4_gemv inner loop:

```
objdump -d --start-address=0x76800 --stop-address=0x77400 \
  build_elbrus/examples/gguf/test_gguf_inference
```

10. Приложение Б: артефакты в репозитории

Путь	Содержимое
torch/io/q8_soa_repack.h	
torch/io/gguf_model.h	
c10/util/ThreadPool.h	
c10/util/Futex.h	
torch/distributed/ddp.cpp	
scripts/run_tp_elbrus.sh	
vliw_mission/e2k_vnni/q8_soa4_gemm4_microbench.c	
vliw_mission/e2k_vnni/q4_soa4_microbench.c	
vliw_mission/round5_lossless_13/RESULTS.md	
vliw_mission/round5_lossless_13/dr_response.md	
BENCH_ELBRUS.md	
JOURNAL.md	

github.com/barometech/PromeTorch
be8fc9d